



Design Review



Nome del gruppo: **Synergo Unit** - Gruppo 20

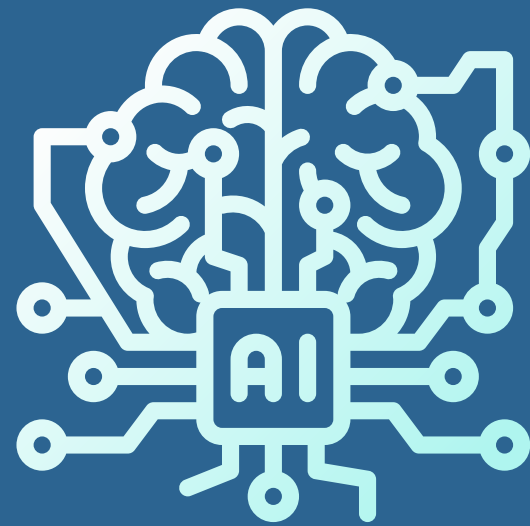
Capitolato C6 - Zucchetti

Revisione PB: 2026/09/xx

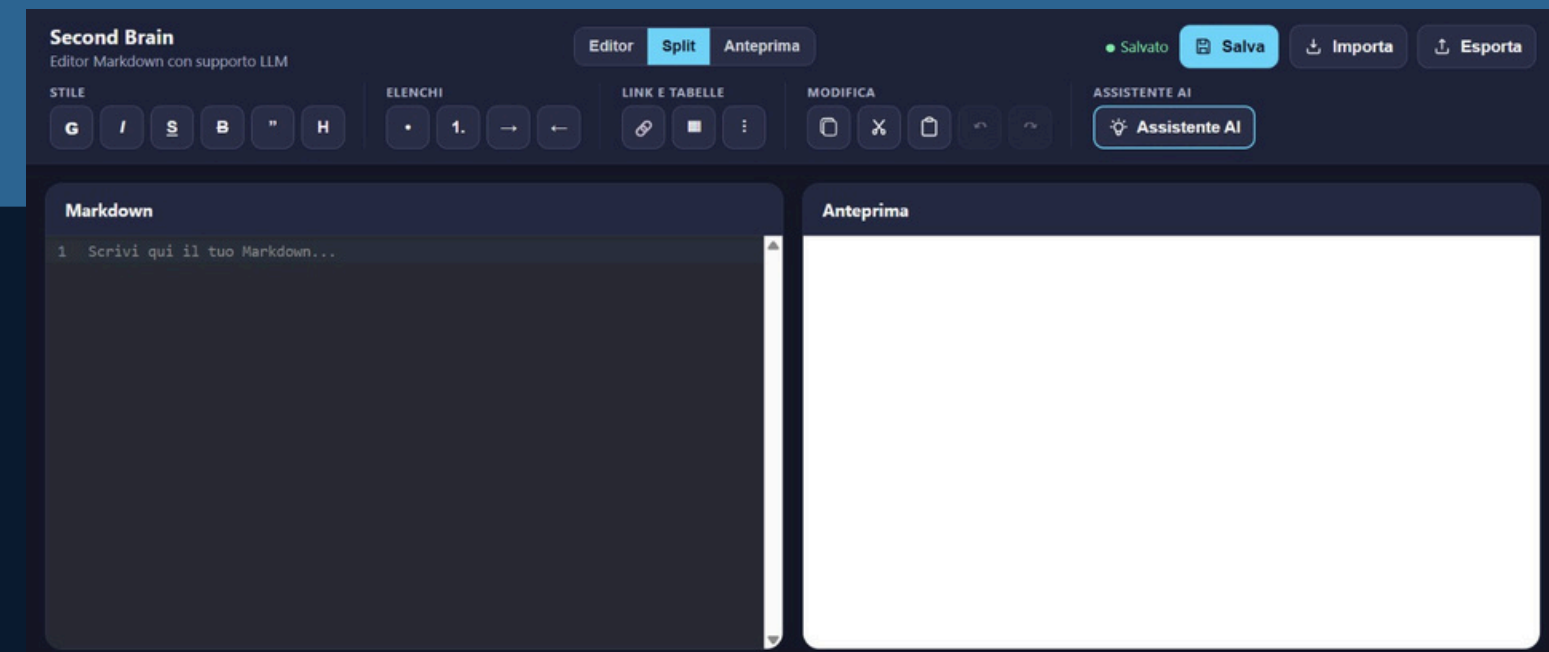
Università degli Studi di Padova
Corso di Ingegneria del Software a.a. 2025-2026

Sofia De Blasi, Francesco Marcon, Filippo Idiometri, Armando Moda Scarati,
Anton Ricardo Rupi, Roberto Mariano Doroftei

Cos'è Second Brain

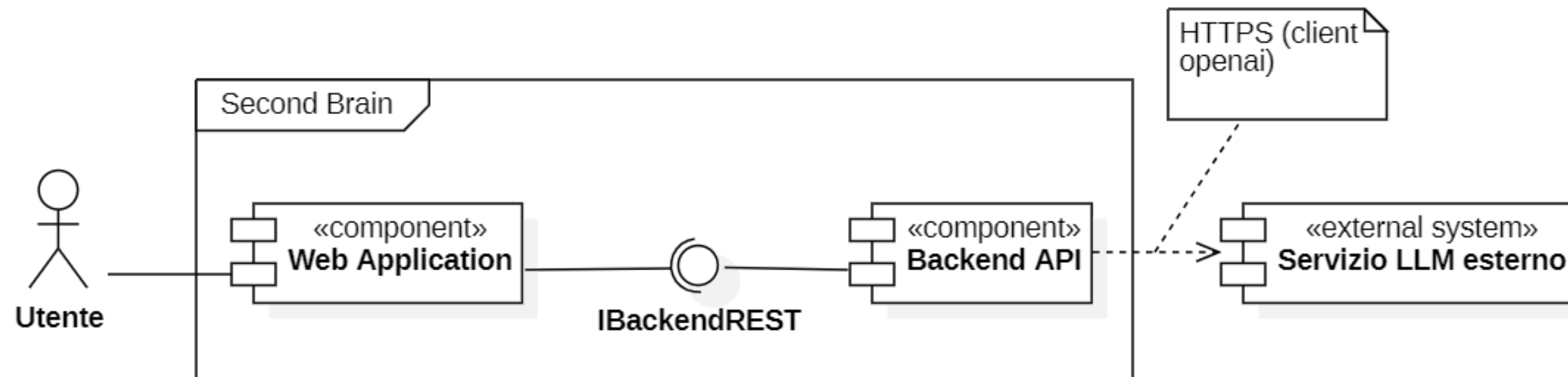


- Editor Markdown web con assistenza AI (LLM)
- Proponente: Zucchetti S.p.A. (capitolato C6)
- Funzioni: formattazione, riassunto/traduzione/riscrittura, Sei Cappelli di De Bono, Distant Writing, export PDF/HTML/JSON
- Utente target: studente, senza competenze tecniche pregresse



Architettura di alto livello

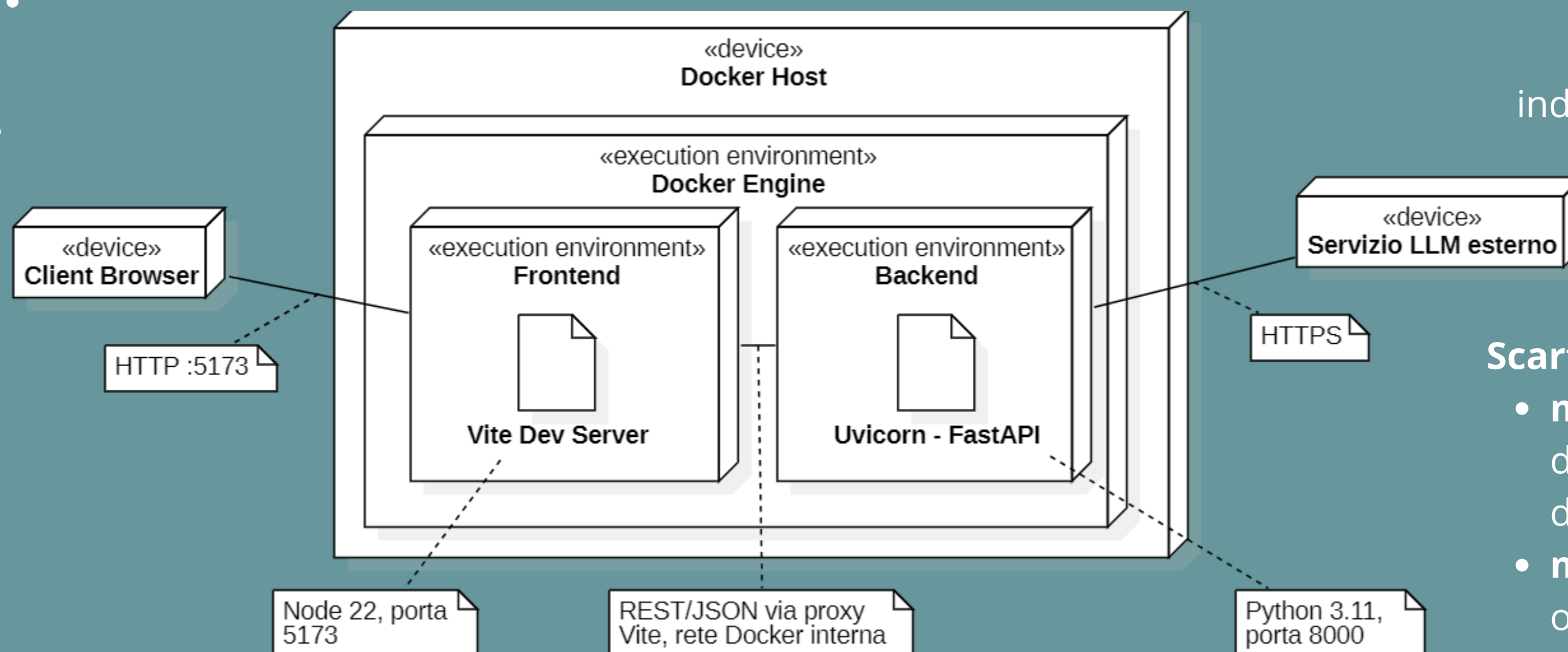
Client-Server, 2 componenti: Web Application (Frontend) + Backend API



- Comunicazione tramite un'unica interfaccia REST (IBackendREST)
- Il Frontend non parla mai direttamente con l'LLM

Deployment: perché due-monoliti containerizzati

2 container Docker indipendenti (frontend Vite, backend Uvicorn/FastAPI)

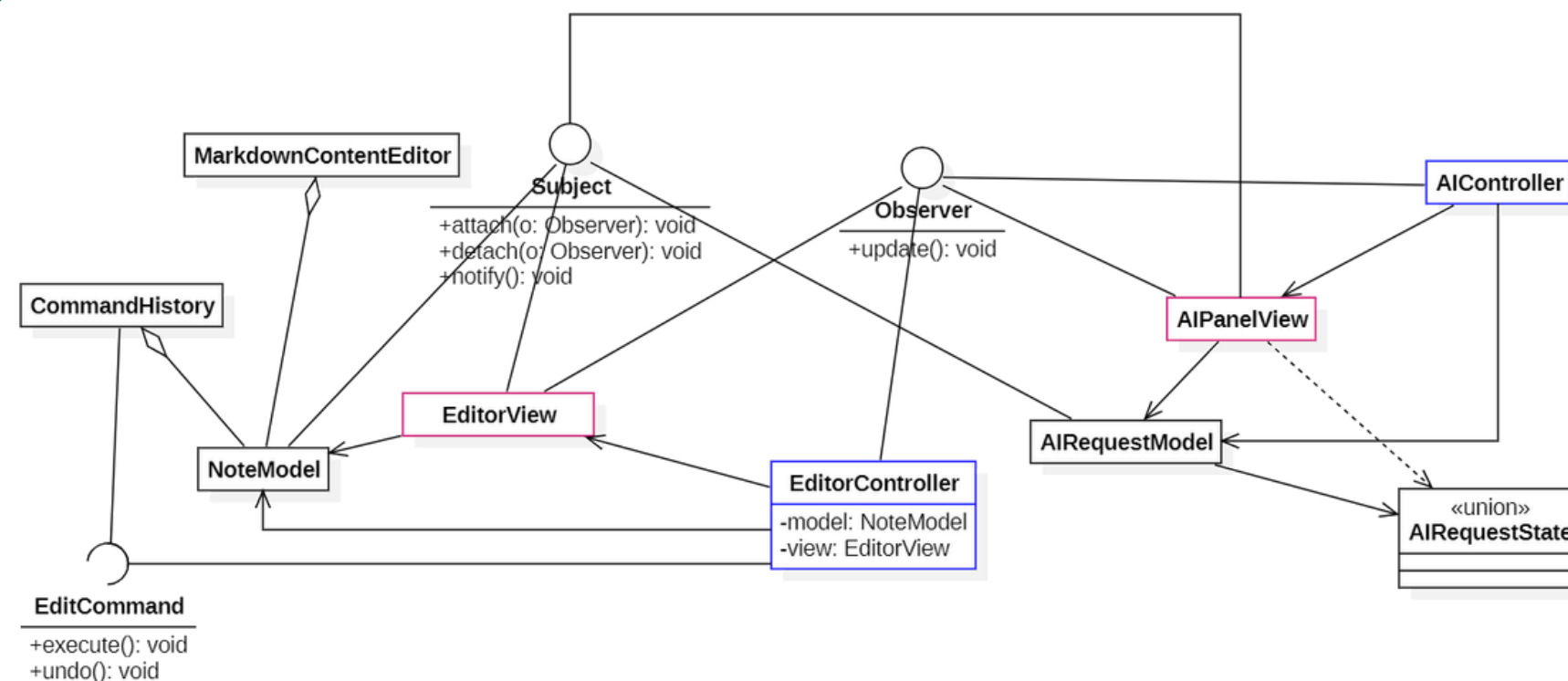


“Two-monolith”: deploy indipendente, nessuna suddivisione interna ulteriore

Scartate:

- **monolite unico** → accoppierebbe deploy statico e deploy con dipendenze esterne
- **microservizi** → complessità di orchestrazione ingiustificata per un dominio applicativo singolo

MVC Pull Model

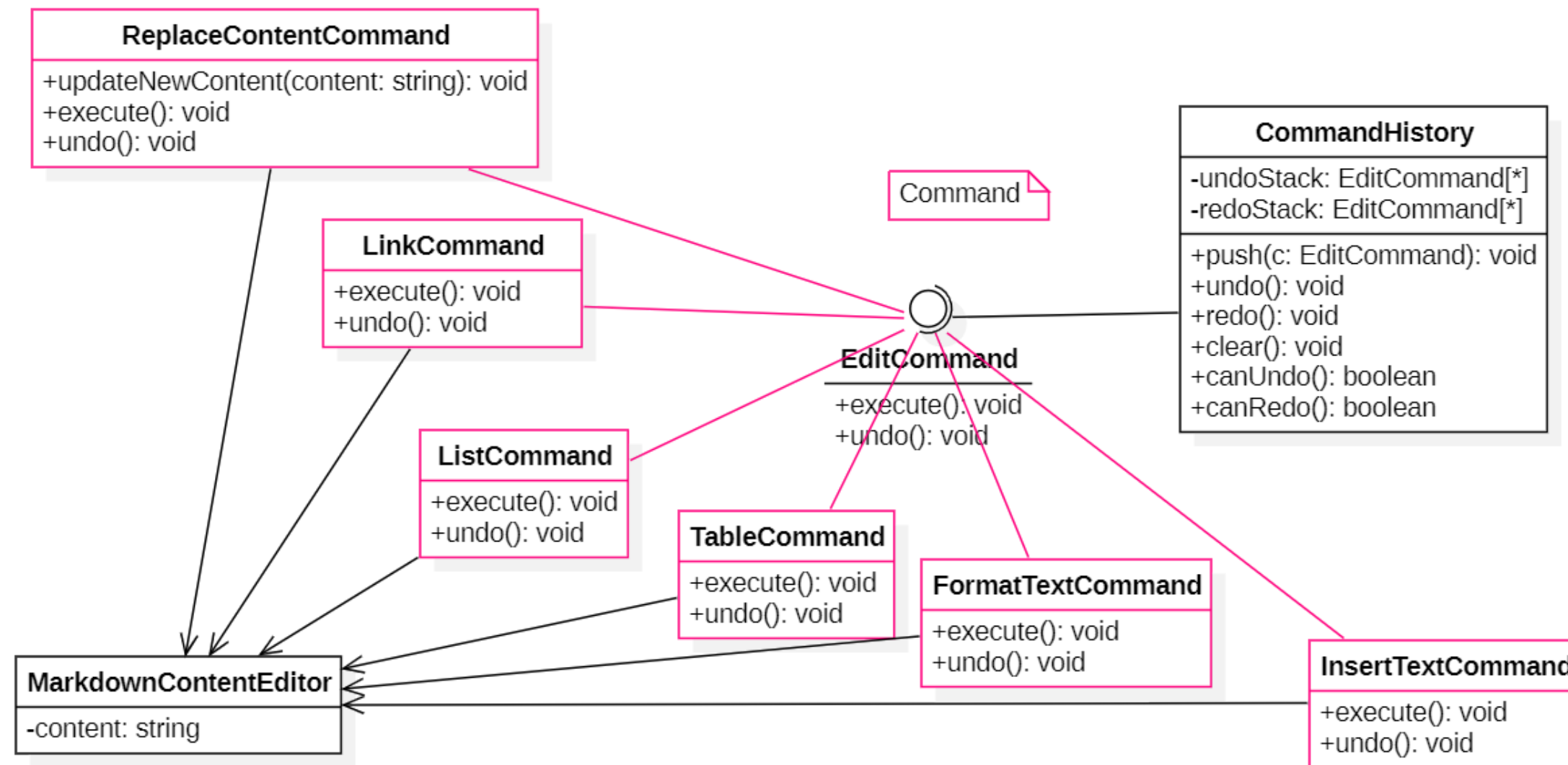


Non una sola triade, ma due, parallele e indipendenti, coerenti con la separazione tra editing e interazioni AI:

- **editing** → NoteModel / EditorController / EditorView
- **AI** → AIRequestModel / AIController / AIPanelView

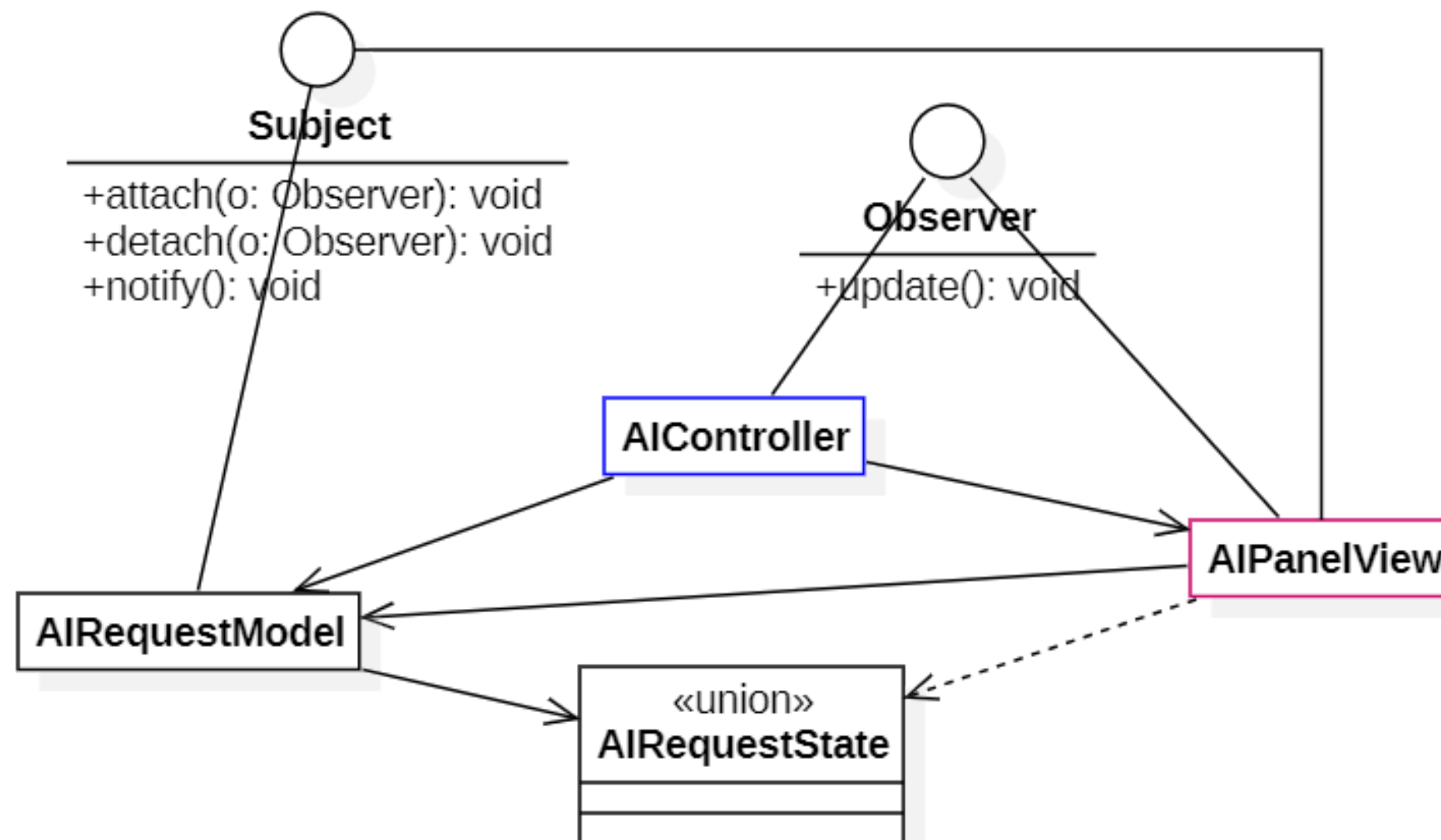
- Model-View-Controller, versione Pull: la View interroga il Model, non riceve dati push
- Model realizza Subject; View realizza sia Subject sia Observer; Controller realizza Observer
- Perché **pull**: il notify() è vuoto, ogni observer decide autonomamente cosa richiedere tramite getX(), invece di ricevere un payload fisso

Observer + Command (editing)



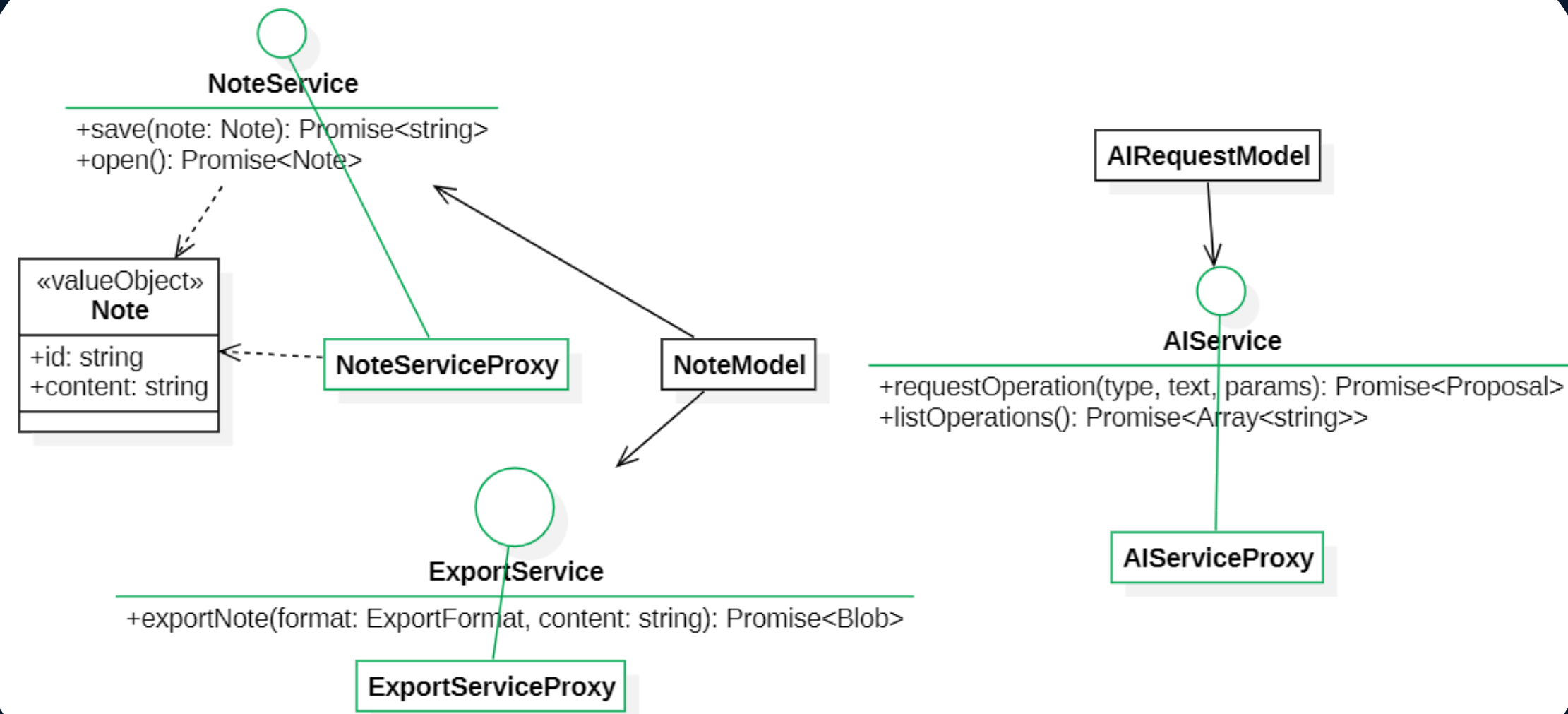
- **NoteModel** mantiene lo stato osservabile della nota, delega la manipolazione testuale a **MarkdownContentEditor**
- Ogni modifica è un **EditCommand** (`execute()/undo()`), eseguito tramite **NoteModel.executeCommand()**
- **CommandHistory**: due pile (undo/redo), uniformi per qualsiasi tipo di modifica

Gestione delle richieste AI (frontend)



- **AIRequestState**: stato come union discriminata → Idle / Processing / ProposalReady / Error
- Non è uno State pattern GoF: nessun comportamento polimorfico per stato, solo dati
- Meccanismo **requestGeneration**: protegge da risposte tardive dopo interruzione/nuova richiesta

Proxy verso il Backend



- Tre interfacce: **NoteService**, **AIService**, **ExportService** → Model dipende solo da queste
- Il Model è ignaro se l'operazione è locale o remota

Implementazioni concrete:

- **NoteServiceProxy** (file system locale + fallback),
- **AIServiceProxy** e **ExportServiceProxy** (REST verso Backend)

Composition Root: come si assembla tutto



Vue non fornisce un contenitore di Dependency Injection equivalente a framework più strutturati



Soluzione:



App.vue è l'unico punto dell'applicazione dove le implementazioni concrete (Proxy) vengono create e passate ai costruttori



Nessun'altra classe del Frontend istanzia mai un Proxy concreto direttamente

```
const markdownEditor = new MarkdownContentEditor("")
const commandHistory = new CommandHistory()
const noteService = new NoteServiceProxy()
const exportService = new ExportServiceProxy("")
const noteModel = new NoteModel(markdownEditor,
commandHistory, noteService, exportService)
```

```
const editorView = new AppEditorView(noteModel, {})
const editorController = new EditorController(noteModel,
editorView)
```

```
const aiService = new AIServiceProxy("")
const aiRequestModel = new AIRequestModel(aiService)
const aiPanelView = new AIPanelView(aiRequestModel)
const aiController = new AIController(aiRequestModel,
aiPanelView, noteModel)
```

Backend: perché Layered e non Esagonale

Opzioni:

Esagonale: più pulita/flessibile per evoluzioni future, ma più complessa da realizzare

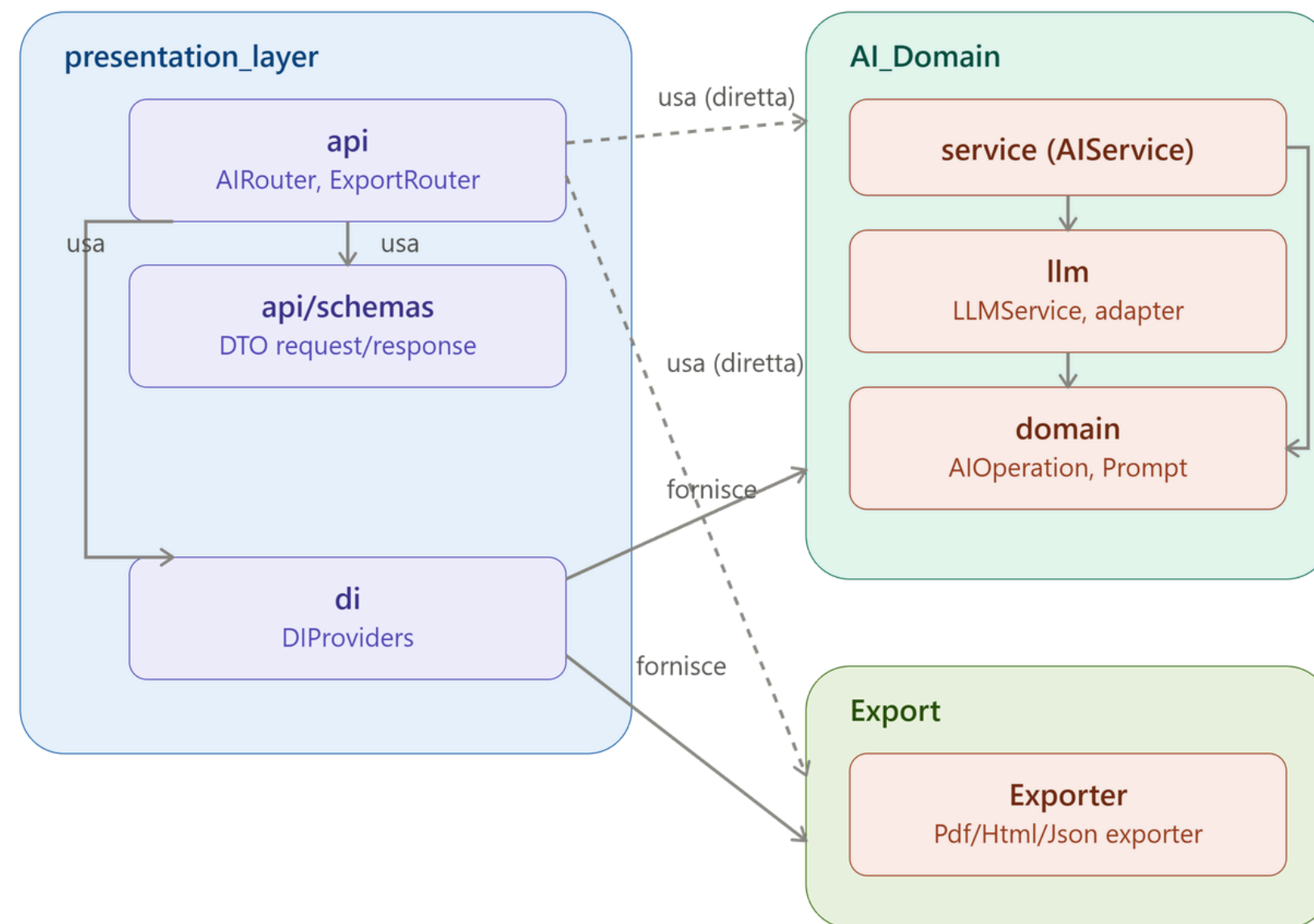
Layer: più naturale, meno onerosa quando non sono previste evoluzioni architetturali e dato il tempo limitato

Decisione finale:

Nel nostro caso: persistenza remota e multi-provider LLM sono opzionali, non implementati → nessuna evoluzione prevista e tempo limitato → Layer

Confronto esplicito con l'azienda proponente (Verbale 30/07/2026, decisione VE-8.1)

Backend



presentation_layer:

Presentazione → router REST, DTO, Dependency Injection

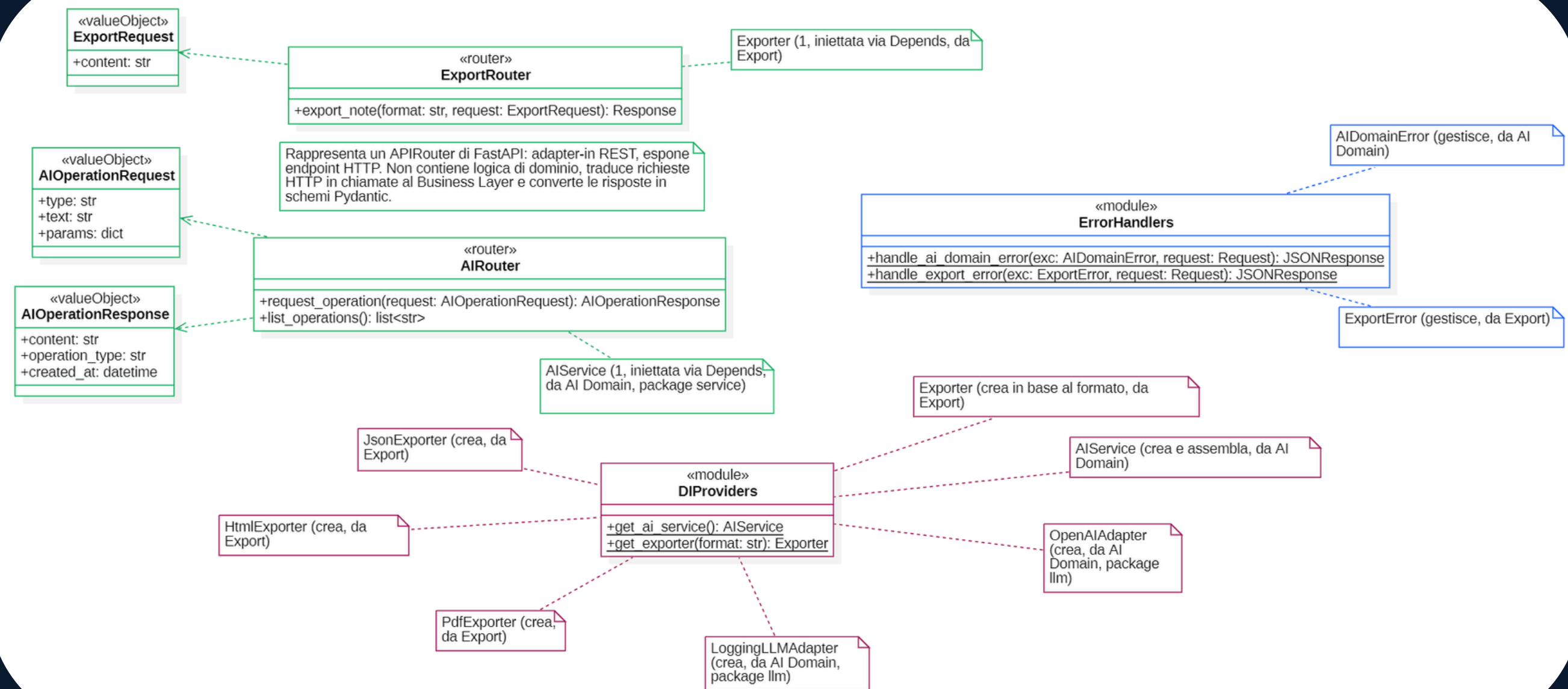
AI_Domain:

Business Logic → prompt building, integrazione LLM, indipendente dal framework web

Export:

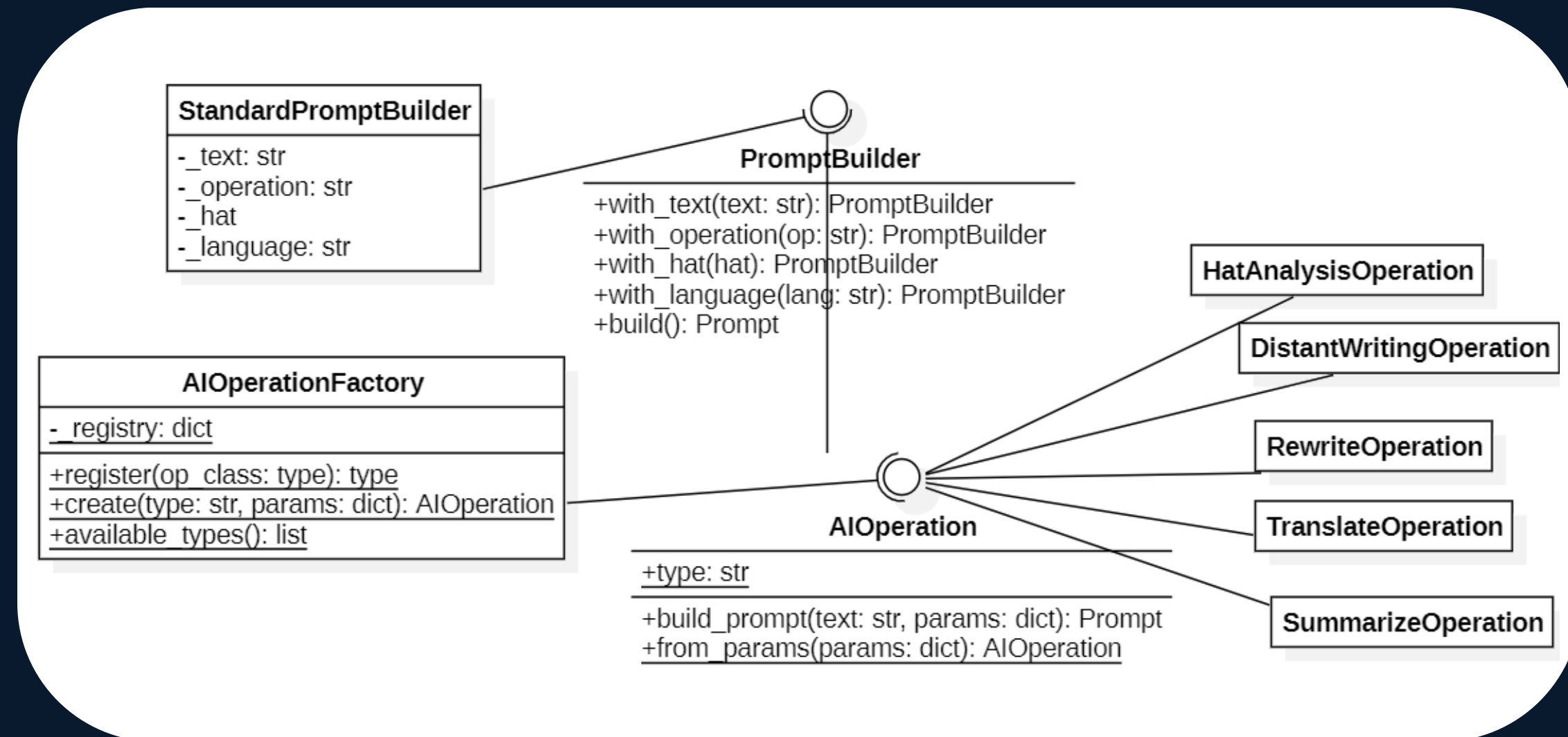
isolato e autosufficiente, non fa parte della catena AI

Livello di Presentazione



- **AIRouter, ExportRouter:** ricevono, validano (DTO Pydantic), delegano → nessuna logica di dominio
- DI nativa di FastAPI: **DIProviders** + Depends
- **ErrorHandlers** centralizzato: traduce eccezioni di dominio in risposte HTTP coerenti

Simple Factory + Strategy nel dominio AI

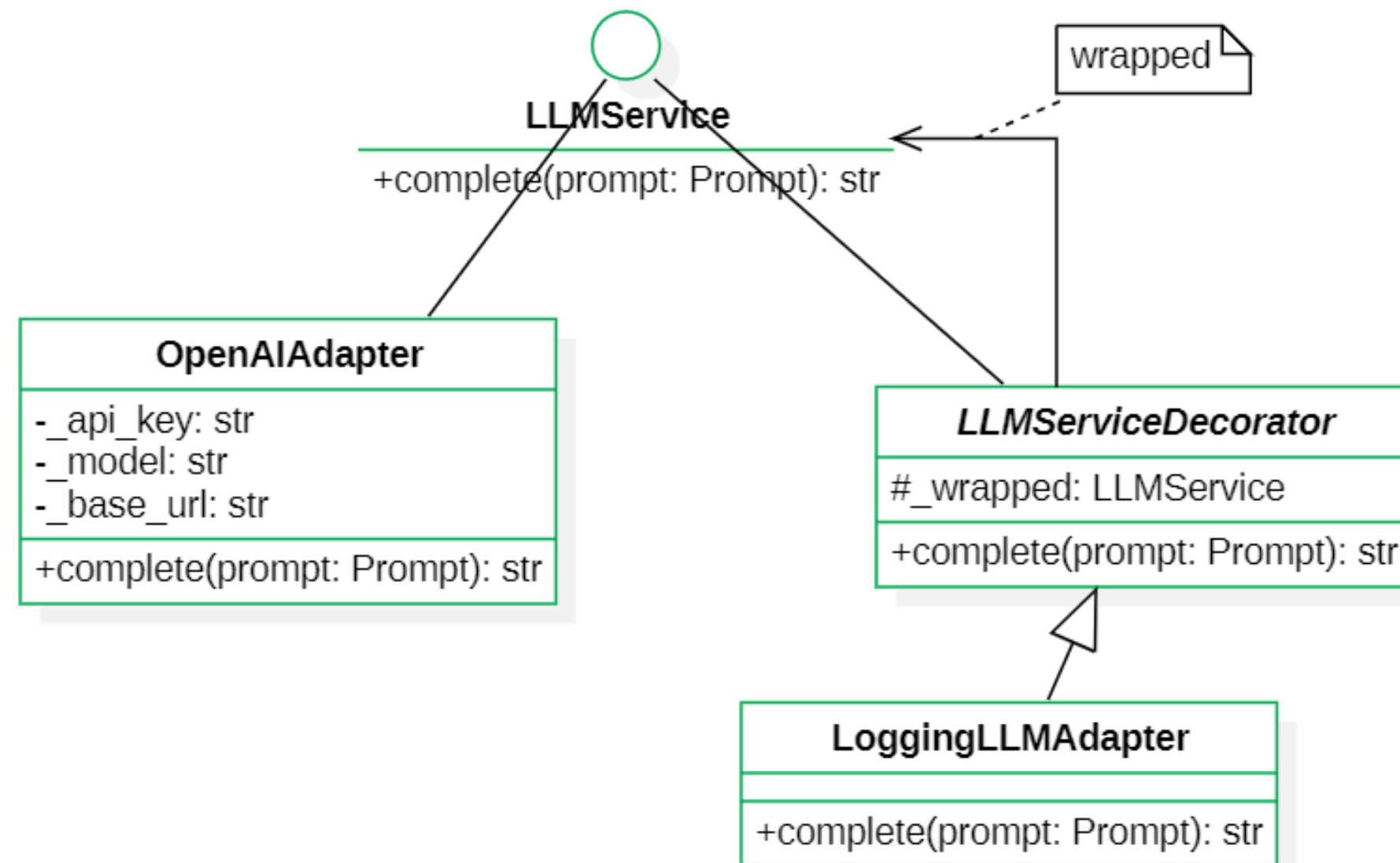


AIOperationFactory: registro auto-registrante, register()/create() → non Factory Method GoF puro

Perché non Factory Method puro?
Raddoppierebbe le classi senza eliminare il bisogno del registro tipo → classe

AIOperation: interfaccia Strategy, build_prompt() polimorfico
→ ogni operazione concreta implementa la propria logica

Decorator + Adapter per LLMService



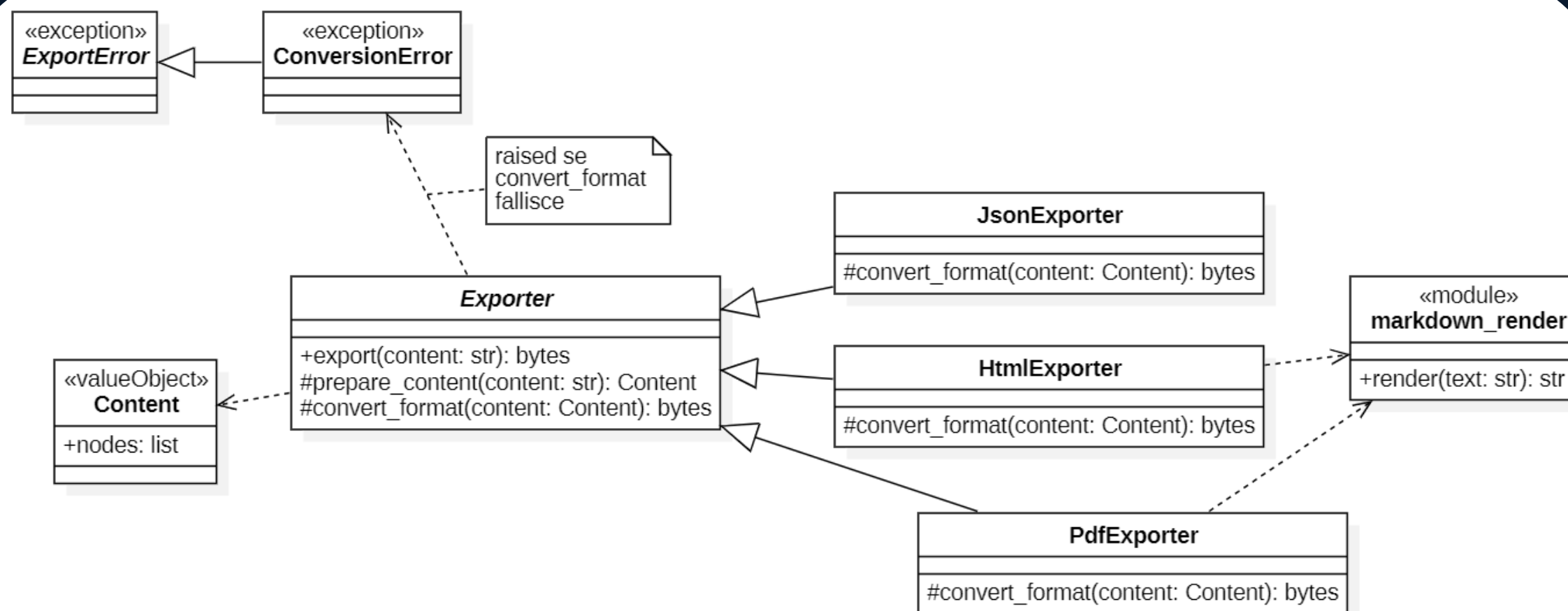
LLMServiceDecorator: classe astratta, wrappa un LLMService → componibile senza toccare AIService

LoggingLLMAdapter: decorator concreto che aggiunge logging

OpenAIAdapter: adapter verso l'API compatibile OpenAI → isola AIService dal provider concreto (Gemma / ChatGPT-OSS / Qwen)

Export: Template Method

Exporter astratta: fissa `export()` → `_prepare_content()` → `_convert_format()`, con `export()` marcato @final



Sottoclassi (**PdfExporter**, **HtmlExporter**, **JsonExporter**) ridefiniscono solo `_convert_format()`

Differenza da Strategy: qui è la classe base a imporre la sequenza dei passi, non il client a scegliere un algoritmo intero intercambiabile

Stato dei requisiti

Tipo requisiti	Stato
100 requisiti obbligatori	Soddisfatti
5 requisiti desiderabili	Soddisfatti
15 requisiti opzionali	Non Implementati